

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4345

Гаврилов Е.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Реализовать и оптимизировать алгоритм поиска с возвратом.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов). Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Пример

Столешница размера 7×7 может быть построена из 9 обрезков:

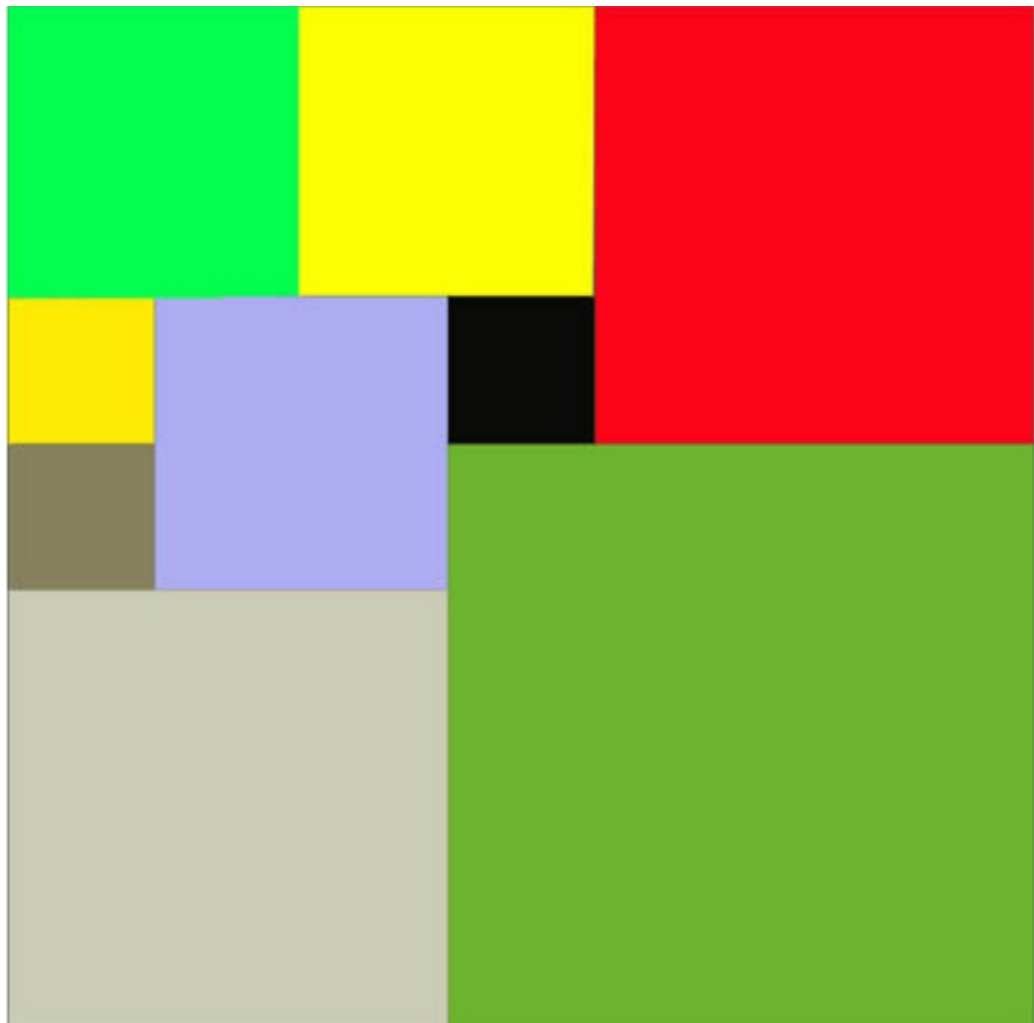


Рисунок 1 – Заполнение квадрата 7×7

Параметры входных данных

Размер столешницы - одно целое число N ($2 \leq N \leq 20$).

Параметры выходных данных

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Sample Input

7

Sample Output

9

```
1 1 2
1 3 2
3 1 1
4 1 1
3 2 2
5 1 3
4 4 4
1 5 3
3 4 1
```

Вариант 1и. Итеративный бэктрекинг. Выполнение на Stepik двух заданий в разделе 2.

Выполнение работы

В ходе работы для оптимизации поиска минимального количества квадратов с целочисленной стороной меньше данного, нужно, чтобы замостить последний, были рассмотрены три оптимизации, соответственно, для трех возможных случаев касемо вводимого числа N :

1 случай – N простое, $N > 2$. Тогда можно оптимизировать только первые шаги алгоритма, которые всегда будут одинаковы: заполнить левый верхний угол квадратом со стороной $(N + 1)/2$, а правый верхний и левый нижний квадратами со стороной $(N - 1)/2$. После этого останется с помощью метода итеративного бэктрекинга заполнить оставшийся квадрат с выколотым квадратом 1×1 в его левом верхнем углу (см. рис. 2). Сам метод заполнения мы опишем далее.

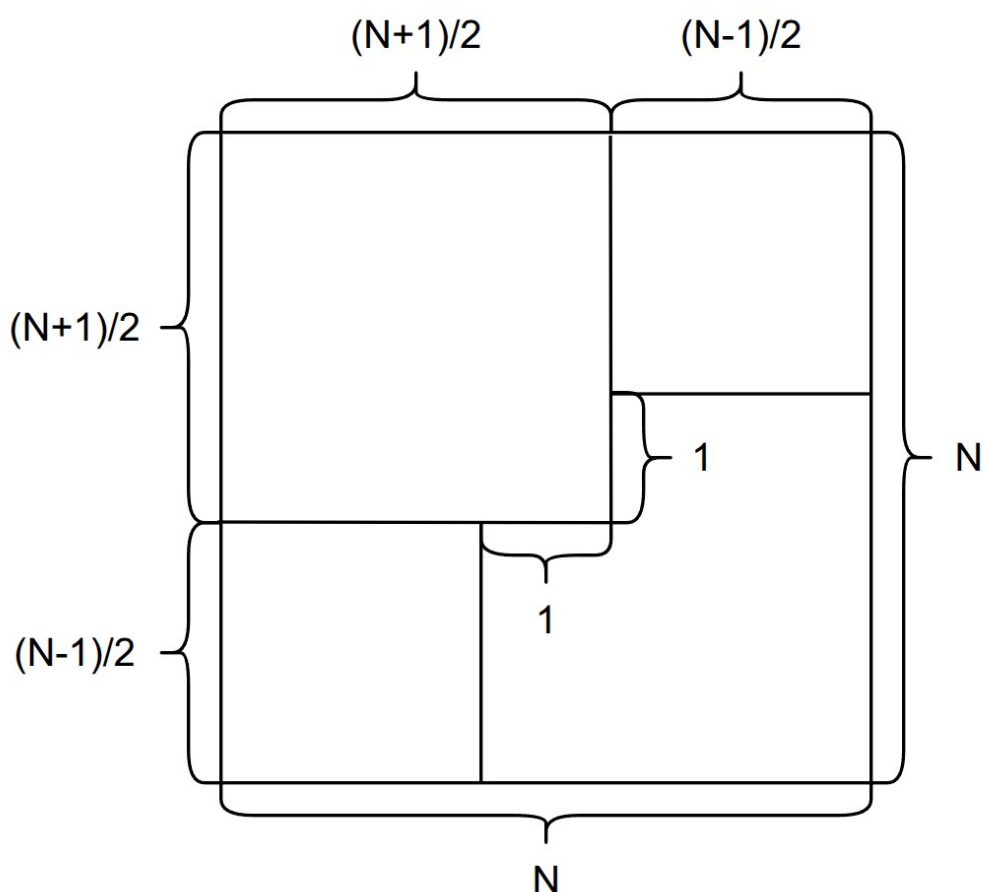


Рисунок 2 – Оптимизация для простого числа

2 случай – N нечетное составное. Тогда можно применить следующую оптимизацию: найти решение для наименьшего простого делителя числа и увеличить его в $N/\{\text{наименьший простой делитель } N\}$ раз.

3 случай – N чётное. Тогда оптимальное заполнение представляет собой четыре квадрата со стороной $N/2$ (см. рис. 4).

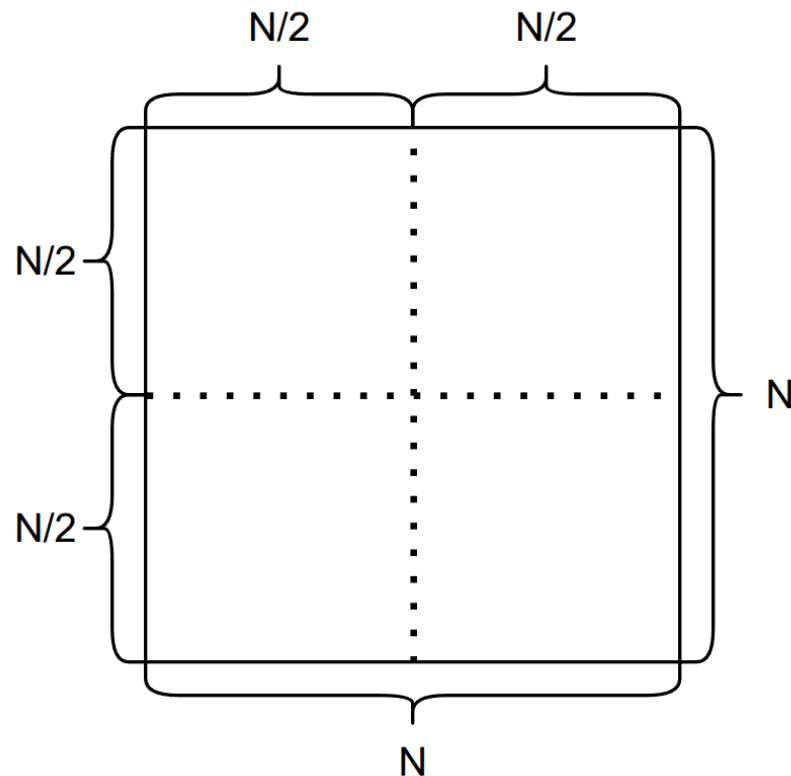


Рисунок 4 – Оптимизация для четного числа

Алгоритм заполнения будет выглядеть следующим образом:

1. Пока есть свободные места и текущее количество квадратов + 1 меньше лучшего:
 - Найти самую верхнюю левую свободную ячейку
 - Определить максимальный размер квадрата, который можно разместить в этой позиции
 - Добавить квадрат максимального возможного размера в текущее разбиение
 - Увеличить счетчик текущего количества квадратов на 1
 - Если доска заполнена полностью, перейти к шагу 3

2. Если текущее количество квадратов + 1 стало больше или равно лучшему количеству прервать текущую ветку поиска (отсечение)
3. Если доска полностью заполнена:
 - Если текущее количество квадратов меньше лучшего:
 - Сохранить текущее разбиение как новое лучшее решение
 - Обновить лучшее количество квадратов
4. Возврат назад:
 - Убрать последний добавленный квадрат из разбиения
 - Уменьшить счетчик текущего количества квадратов на 1
 - Если убранный квадрат имел размер больше 1:
 - Уменьшить размер этого квадрата на 1
 - Добавить квадрат уменьшенного размера в ту же позицию
 - Увеличить счетчик текущего количества квадратов на 1
 - Перейти к шагу 1
 - Иначе (размер = 1):
 - Продолжать убирать квадраты с конца разбиения
 - Уменьшать счетчик текущего количества квадратов
 - Пока не найдем квадрат с размером больше 1 или разбиение не станет пустым
 - Если разбиение пустое (все квадраты убраны) алгоритм завершен, лучшее решение найдено
 - Иначе (размер > 1):
 - Уменьшить размер этого квадрата на 1
 - Добавить квадрат уменьшенного размера обратно
 - Увеличить счетчик текущего количества квадратов
 - Перейти к шагу 1

Ниже приведено описание реализованных структур и методов.

Описание структуры Square

Структура Square представляет размещённый на доске квадрат.

Содержит три целочисленных поля:

- `x` – координата верхнего левого угла квадрата по вертикали
- `y` – координата верхнего левого угла квадрата по горизонтали
- `size` – длина стороны квадрата

Описание структуры State

Структура State представляет один элемент в стеке итеративного поиска с возвратом. Содержит поля:

- `currentSquares` – срез структур Square, хранящий квадраты, размещённые к текущему моменту в этой ветке поиска
- `count` – количество размещённых квадратов в данном состоянии
- `row` – координата строки текущей найденной пустой ячейки, для которой перебираются варианты
- `col` – координата колонки текущей найденной пустой ячейки
- `sizeIndex` – индекс текущего размера в срезе `sizes`, который будет попробован следующим.
- `sizes` – срез целых чисел, список размеров квадратов (в порядке убывания), которые можно попробовать поместить в позицию (`row`, `col`).
- `board` – срез целых чисел, копия состояния доски (битовой маски) на момент этого элемента стека

Описание структуры Solver

Структура Solver инкапсулирует состояние, необходимое для решения задачи упаковки квадратов. Содержит поля:

- `boardSize` – размер стороны квадратной доски
- `bestCount` – наименьшее найденное количество квадратов для покрытия доски
- `bestSquares` – срез структур Square, хранящий координаты и размеры квадратов для лучшего найденного решения

- board – срез целых чисел, представляющий доску в виде битовой маски занятых клеток. Каждый элемент среза соответствует строке доски, а установленные биты внутри элемента – занятым колонкам в этой строке

Описание функции NewSolver(n int) *Solver

n – размер доски. Функция создаёт и возвращает указатель на новый экземпляр структуры Solver. В созданном экземпляре поле boardSize инициализируется значением n.

Описание метода place(row, col, size int, add bool)

Метод принадлежит структуре Solver и используется для добавления или удаления квадрата с доски. На вход подаются координаты верхнего левого угла (row, col), размер стороны size и булево значение add.

В начале метод выводит в консоль отладочное сообщение о выполняемом действии: размещении или удалении квадрата с указанием его размера и координат.

Затем вычисляется битовая маска для строки: $((1 \ll \text{size}) - 1) \ll \text{col}$, которая создаёт последовательность из size единичных битов, сдвинутых влево на col позиций. Далее в цикле for по i от row до row+size-1 происходит обход всех строк, которые занимает квадрат. Для каждой строки i выполняется проверка: если add истинно, то к s.board[i] применяется побитовое ИЛИ (|=) с маской, устанавливая биты занятых клеток в 1. Иначе применяется побитовое И НЕ (&^=) с маской, сбрасывая биты занятых клеток в 0. После изменения доски вызывается метод printBoard() для вывода результирующего состояния.

Описание метода findEmpty() (int, int)

Метод принадлежит структуре Solver и находит самую верхнюю, а среди них самую левую пустую клетку на доске. В начале выводится сообщение о начале поиска. Вычисляется fullMask – маска полностью заполненной строки: $(1 \ll \text{s.boardSize}) - 1$. Затем в цикле for по row от 0 до s.boardSize-1 для каждой строки проверяется, не равна ли она fullMask. Если строка заполнена не полностью, то в цикле for переменная col инициализируется нулём и

увеличивается, пока бит $(s.board[row] \gg col) \& 1$ равен 1. Как только найден нулевой бит, выводятся координаты найденной пустой клетки, и они возвращаются. Если все строки полностью заполнены, выводится сообщение, что пустая клетка не найдена, и возвращается -1, -1.

Описание метода `maxSquareSize(row, col int) int`

Метод принадлежит структуре Solver и определяет размер наибольшего квадрата, который можно разместить, начиная с позиции (row, col). На вход подаются координаты row и col. В начале выводится сообщение о поиске максимального квадрата для заданных координат. Вычисляется начальное ограничение maxSize как минимум из $s.boardSize - row$ и $s.boardSize - col$. Далее в цикле for по size от 1 до maxSize для каждого размера формируется маска $((1 \ll size) - 1) \ll col$ и во вложенном цикле по i от row до row+size-1 проверяется, пересекается ли маска с уже занятыми клетками в строке i (проверка $s.board[i] \& mask \neq 0$). Если пересечение обнаружено, выводится сообщение, что наибольший квадрат имеет размер size-1, и это значение возвращается. Если цикл завершился без нахождения пересечений, возвращается maxSize.

Описание метода `copyBoard() []int`

Метод принадлежит структуре Solver. Создает и возвращает полную копию текущего состояния доски. Выделяется новый срез целых чисел длиной s.boardSize, и в него копируются элементы из s.board.

Описание метода `restoreBoard(savedBoard []int)`

Метод принадлежит структуре Solver. На вход подаётся срез целых чисел savedBoard – сохранённое ранее состояние доски. Восстанавливает доску, копируя элементы из savedBoard обратно в s.board.

Описание метода `backtrack(initialSquares []Square, initialCount int)`

Метод принадлежит структуре Solver и выполняет итеративный поиск с возвратом для нахождения оптимальной упаковки квадратов, используя явный

стек. На вход подаются через начальных квадратов `initialSquares` и их количество `initialCount`.

В начале инициализируется пустой через `stack` структур `State`, который будет использоваться как стек. Вызывается `s.findEmpty()` для поиска первой пустой ячейки. Если доска не заполнена (`row != -1`), вычисляется максимальный размер квадрата для этой позиции и формируется через `sizes`, содержащий размеры от максимального до 1 в порядке убывания.

Создаётся начальное состояние `initialState`, куда помещаются копия переданных начальных квадратов, их количество, координаты найденной пустой ячейки, обнулённый `sizeIndex`, список размеров `sizes` и копия доски. Это состояние добавляется в стек.

Далее выводится отладочная информация о вызове поиска с возвратом: текущая глубина, количество квадратов, их список и лучшее найденное количество.

Затем запускается основной цикл `for len(stack) > 0`. На каждой итерации анализируется верхнее состояние стека (`current`).

Если `current.row == -1` (доска полностью заполнена), выводится сообщение о найденном решении. Если `current.count < s.bestCount`, обновляется лучший результат (`s.bestCount` и `s.bestSquares`). Затем состояние выталкивается из стека, и если стек не пуст, доска восстанавливается из состояния нового верхнего элемента.

Иначе, если `current.count+1 >= s.bestCount` (срабатывает условие отсечения), выводится сообщение об отсечении ветви, состояние выталкивается, и доска восстанавливается.

Иначе, если `current.sizeIndex >= len(current.sizes)` (все размеры для данной позиции перебраны), состояние выталкивается, и доска восстанавливается.

В противном случае из `current.sizes` берётся очередной размер `size` по индексу `sizeIndex`, после чего `sizeIndex` увеличивается. Текущая доска

восстанавливается из `current.board`. Вызывается `s.place(current.row, current.col, size, true)` для размещения квадрата. Затем ищется следующая пустая клетка (`nextRow, nextCol`). Если она найдена, формируется новый список размеров `nextSizes` в порядке убывания. Создаётся новый список квадратов `newSquares`, включающий все квадраты из `current.currentSquares` плюс новый размещённый квадрат. Формируется и добавляется в стек новое состояние `newState` с обновлёнными данными. Доска восстанавливается из сохранённой копии `savedBoard`, чтобы продолжить перебор других вариантов из того же родительского состояния.

Описание функции `solveEven()` (`int`, `[]Square`)

Метод принадлежит структуре `Solver` и возвращает оптимальное решение для доски чётного размера. Вычисляется $half = s.boardSize / 2$. Возвращается число 4 и срез из четырёх квадратов размером `half`, размещённых в углах: `(1,1)`, `(1, half+1)`, `(half+1, 1)`, `(half+1, half+1)`.

Описание функции `scaleSolution(count int, squares []Square, factor int)` (`int`, `[]Square`)

Функция масштабирует решение для составного размера доски. На вход подаются количество квадратов `count`, срез исходных квадратов `squares` и целочисленный множитель `factor`. Создаётся новый срез `scaled` длиной `count`. В цикле по всем элементам `squares` координаты и размер каждого квадрата умножаются на `factor` с корректировкой координат (формула $(sq.x-1)*factor + 1$). Возвращается исходное количество и масштабированный срез.

Описание функции `solve()` (`int`, `[]Square`)

Метод принадлежит структуре `Solver` и управляет процессом поиска оптимальной упаковки для установленного `boardSize`. В начале выводится сообщение о поиске решения.

Если `s.boardSize` чётное, выводится сообщение о тривиальности решения и вызывается `s.solveEven()`, результат которого возвращается.

Если `s.boardSize` составное (проверяется через вызов `smallestPrimeFactor`, возвращающий значение меньше `s.boardSize`), выводится сообщение о решении для наименьшего простого множителя с последующим масштабированием. Создаётся новый экземпляр `Solver` для этого простого множителя, рекурсивно вызывается его метод `solve()`, а полученное решение масштабируется через `scaleSolution`.

Если `s.boardSize` простое, выводится сообщение о полном поиске. Инициализируется `s.bestCount` заведомо большим значением, выделяется память под `s.board`. Для доски больше 2 применяется эвристика с размещением трёх начальных квадратов: вычисляются $size1 = (s.boardSize + 1) / 2$ и $size2 = (s.boardSize - 1) / 2$, они размещаются в позициях $(0,0)$, $(0,size1)$ и $(size1,0)$. Эти квадраты передаются в `s.backtrack` вместе с начальным количеством 3. Иначе `s.backtrack` вызывается с пустыми начальными данными. По завершении возвращаются `s.bestCount` и `s.bestSquares`.

Описание функции `smallestPrimeFactor(n int) int`

На вход подаётся целое число `n`. Функция находит и возвращает наименьший простой множитель `n`. В цикле `for i := 2; i*i <= n; i++` проверяется, делится ли `n` на `i` без остатка. Если делится, выводится сообщение с найденным множителем, и он возвращается. Если цикл завершён, а делитель не найден, выводится сообщение, что `n` само является своим наименьшим простым множителем, и возвращается `n`.

Описание метода `printBoard()`

Метод принадлежит структуре `Solver` и выводит текущее состояние доски в консоль в текстовом виде. Если `s.board` пуст, выводится сообщение о неинициализированной доске. Иначе во вложенных циклах по `row` и `col` для каждой клетки проверяется бит $(s.board[row] \gg col) \& 1$. Если бит установлен, выводится символ "■", иначе "□". После каждой строки осуществляется перевод каретки, а после всей доски – дополнительный перевод строки.

Описание функции main()

Точка входа в программу. Считывает из стандартного ввода целое число boardSize. Создает новый экземпляр Solver для этого размера доски. Вызывает метод solve() для поиска оптимальной упаковки, получая количество квадратов count и список squares. Выводит в стандартный вывод count, а затем для каждого квадрата из squares выводит его координаты x, y и размер size в одной строке через пробел.

Сложность алгоритма

по времени: $O(1)$ для четных N , $O(C^{N^{2/4}})$, где C – среднее количество вариантов размера квадрата (деление N^2 на 4 за счет оптимизации – сокращается площадь квадрата, N^2 за счет перебора по всей площади малого квадрата).

по памяти: $O(1)$ для четных N , $O(N^2)$ для нечетных N .

Исходный код программы см. в приложении А.

Результаты тестирования программы см. в Таблице 1.

Таблица 1 – Тестирование программы

№	Ввод	Вывод	Комментарий
1	7	9 1 1 4 1 5 3 5 1 3 4 5 2 4 7 1 5 4 1 5 7 1 6 4 2 6 6 2	Тестовый пример.
2	8	4 1 1 4 1 5 4 5 1 4 5 5 4	Четное число – считается мгновенно.
3	21	6 1 1 14 1 15 7	Вывод такой же как для 3, но масштабированный, считается с той же скоростью.

		15 1 7 8 15 7 15 8 7 15 15 7	
4	3	3 6 1 1 2 1 3 1 3 1 1 2 3 1 3 2 1 3 3 1	Маленькое простое число.
5	43	16 1 1 22 1 23 21 23 1 21 22 23 2 22 25 8 22 33 5 22 38 6 23 22 1 24 22 3 27 22 3 27 33 3 27 36 1 27 37 1 28 36 8 30 22 14 36 36 8	Большое простое число.

Вывод

Реализована программа для нахождения минимального количества квадратов стороны $< N$, нужных чтобы покрыть площадь квадрата стороны N . Для этого применены три оптимизации и алгоритм поиска с возвратом.

Для четных чисел (значений стороны квадрата) ответ получается практически мгновенно за константное время, для простых сложность по времени растет экспоненциально, для составных – растет также экспоненциально, но зависит от наименьшего простого числа, на которое делится данное составное число. Таким образом, наихудший случай для алгоритма – это большое простое число или составное, являющееся произведением больших простых чисел, наилучший – составное число, которое делится на малое простое, частный случай – чётные числа, для которых время работы алгоритма $O(1)$.

Реализация описанного алгоритма протестирована вручную, в результате тестирования ошибок не выявлено.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.go

```
package main

import (
    "fmt"
)

// Square represents a placed square on the board
type Square struct {
    x, y, size int
}

// State represents a single frame in the backtracking stack
type State struct {
    currentSquares []Square
    count          int
    row            int
    col            int
    sizeIndex      int    // current index in the sizes list we're
trying
    sizes          []int // list of sizes to try at this position
    board          []int // copy of board state at this frame
}

// Solver encapsulates the state for solving the square packing
problem
type Solver struct {
    boardSize  int
    bestCount  int
    bestSquares []Square
    board      []int // bitmask representation of occupied cells
}

// NewSolver creates a new Solver instance for a given board size
func NewSolver(n int) *Solver {
    return &Solver{
        boardSize: n,
    }
}

// place adds or removes a square from the board using bitmask
operations
func (s *Solver) place(row, col, size int, add bool) {
    if add {
        fmt.Printf("\nPlacing a square of size %d to position (%d,
%d)\n", size, row, col)
    } else {
        fmt.Printf("\nRemoving a square of size %d from position
(%d, %d)\n", size, row, col)
    }
}
```

```

        mask := ((1 << size) - 1) << col
        for i := row; i < row+size; i++ {
            if add {
                s.board[i] |= mask
            } else {
                s.board[i] ^= mask
            }
        }

        fmt.Println("Resulting board:")
        s.printBoard()
    }

    // findEmpty locates the topmost-leftmost empty cell
    func (s *Solver) findEmpty() (int, int) {
        fmt.Println("Finding topmost-leftmost empty cell...")

        fullMask := (1 << s.boardSize) - 1
        for row := 0; row < s.boardSize; row++ {
            if s.board[row] != fullMask {
                col := 0
                for (s.board[row]>>col)&1 == 1 {
                    col++
                }
                fmt.Printf("Empty cell found on position (%d, %d)\n",
row, col)
                return row, col
            }
        }
        fmt.Println("Empty cell not found")
        return -1, -1
    }

    // maxSquareSize determines the largest square that can fit at (row,
col)
    func (s *Solver) maxSquareSize(row, col int) int {
        fmt.Printf("\nFinding the largest square that can fit at (%d, %d)
\n", row, col)

        maxSize := s.boardSize - row
        if s.boardSize-col < maxSize {
            maxSize = s.boardSize - col
        }

        for size := 1; size <= maxSize; size++ {
            mask := ((1 << size) - 1) << col
            for i := row; i < row+size; i++ {
                if s.board[i]&mask != 0 {
                    fmt.Printf("Largest square at (%d, %d) has size
%d\n", row, col, size-1)
                    return size - 1
                }
            }
        }
    }
}

```

```

        fmt.Printf("The largest square on (%d, %d) is of size %d\n", row,
col, maxSize)
        return maxSize
    }

    // copyBoard creates a deep copy of the current board
    func (s *Solver) copyBoard() []int {
        newBoard := make([]int, s.boardSize)
        copy(newBoard, s.board)
        return newBoard
    }

    // restoreBoard restores the board from a saved state
    func (s *Solver) restoreBoard(savedBoard []int) {
        copy(s.board, savedBoard)
    }

    // backtrack performs iterative search for optimal square packing
    using a stack
    func (s *Solver) backtrack(initialSquares []Square, initialCount int)
    {
        // Stack of states to process
        stack := []State{}

        // Initial state setup
        row, col := s.findEmpty()
        sizes := []int{}
        if row != -1 {
            maxSize := s.maxSquareSize(row, col)
            sizes = make([]int, maxSize)
            for i := 0; i < maxSize; i++ {
                sizes[i] = maxSize - i // descending order
            }
        }

        initialState := State{
            currentSquares: make([]Square, len(initialSquares)),
            count:            initialCount,
            row:              row,
            col:              col,
            sizeIndex:        0,
            sizes:            sizes,
            board:            s.copyBoard(),
        }
        copy(initialState.currentSquares, initialSquares)

        stack = append(stack, initialState)

        fmt.Println("\nBACKTRACK CALL")
        fmt.Printf("Current depth: %d\n", initialCount)
        fmt.Printf("Current squares count: %d\n", len(initialSquares))
        fmt.Print("Current squares: [")
        for i, sq := range initialSquares {
            if i > 0 {
                fmt.Print(", ")
            }
        }
    }

```

```

        fmt.Printf("(%d,%d,%d)", sq.x, sq.y, sq.size)
    }
    fmt.Println("]")
    fmt.Printf("Best count so far: %d\n", s.bestCount)

    for len(stack) > 0 {
        // Peek at the top state
        current := &stack[len(stack)-1]

        // Check if board is completely filled
        if current.row == -1 {
            fmt.Println("\n\tBOARD COMPLETELY FILLED!!!")
            fmt.Printf("Solution found with %d squares\n",
current.count)
            if current.count < s.bestCount {
                fmt.Printf("NEW BEST SOLUTION! Previous best: %d
-> New best: %d\n", s.bestCount, current.count)
                s.bestCount = current.count
                s.bestSquares = make([]Square, current.count)
                copy(s.bestSquares, current.currentSquares)
            } else {
                fmt.Printf("Solution is not better than current
best: %d\n", s.bestCount)
            }
            // Pop this state
            stack = stack[:len(stack)-1]
            if len(stack) > 0 {
                // Restore board from the parent state
                s.restoreBoard(stack[len(stack)-1].board)
            }
            if len(stack) > 0 {
                fmt.Printf("BACKTRACK RETURN (depth %d)\n",
stack[len(stack)-1].count)
            }
            continue
        }

        // Pruning: no need to continue if we already have a better
solution
        if current.count+1 >= s.bestCount {
            fmt.Printf("PRUNING: count+1 (%d) >= bestCount (%d) -
abandoning this branch\n", current.count+1, s.bestCount)
            // Pop this state
            stack = stack[:len(stack)-1]
            if len(stack) > 0 {
                s.restoreBoard(stack[len(stack)-1].board)
            }
            if len(stack) > 0 {
                fmt.Printf("BACKTRACK RETURN (depth %d)\n",
stack[len(stack)-1].count)
            }
            continue
        }

        // If we've tried all sizes for this position, pop the
state

```

```

        if current.sizeIndex >= len(current.sizes) {
            stack = stack[:len(stack)-1]
            if len(stack) > 0 {
                s.restoreBoard(stack[len(stack)-1].board)
            }
            if len(stack) > 0 {
                fmt.Printf("BACKTRACK RETURN (depth %d)\n",
stack[len(stack)-1].count)
            }
            continue
        }

        // This is where we need to simulate the recursive call
structure
        // Print the backtrack call header for each new position
we're exploring
        if current.sizeIndex == 0 {
            fmt.Println("\nBACKTRACK CALL")
            fmt.Printf("Current depth: %d\n", current.count)
            fmt.Printf("Current squares count: %d\n",
len(current.currentSquares))
            fmt.Print("Current squares: [")
            for i, sq := range current.currentSquares {
                if i > 0 {
                    fmt.Print(", ")
                }
                fmt.Printf("(%d,%d,%d)", sq.x, sq.y, sq.size)
            }
            fmt.Println("]")
            fmt.Printf("Best count so far: %d\n", s.bestCount)
        }

        // Get the current size to try
        size := current.sizes[current.sizeIndex]
        current.sizeIndex++

        fmt.Printf("\n--- Trying size %d at position (%d, %d) ---
\n", size, current.row, current.col)

        // Save current board before placing
        savedBoard := s.copyBoard()
        s.restoreBoard(current.board)

        // Place the square
        s.place(current.row, current.col, size, true)

        // Find next empty cell for the recursive call
        nextRow, nextCol := s.findEmpty()
        nextSizes := []int{}
        if nextRow != -1 {
            nextMaxSize := s.maxSquareSize(nextRow, nextCol)
            nextSizes = make([]int, nextMaxSize)
            for i := 0; i < nextMaxSize; i++ {
                nextSizes[i] = nextMaxSize - i
            }
        }

```

```

        // Create new squares list
        newSquares := make([]Square, len(current.currentSquares)+1)
        copy(newSquares, current.currentSquares)
        newSquares[len(current.currentSquares)] = Square{
            x:      current.row + 1,
            y:      current.col + 1,
            size: size,
        }

        // Push new state onto stack (this represents the recursive
call)
        newState := State{
            currentSquares: newSquares,
            count:          current.count + 1,
            row:            nextRow,
            col:            nextCol,
            sizeIndex:      0,
            sizes:          nextSizes,
            board:          s.copyBoard(),
        }
        stack = append(stack, newState)

        // Restore the board for the current state (it will be
restored again when we come back)
        s.restoreBoard(savedBoard)
    }
}

// solveEven handles even-sized boards with optimal solution (4
squares)
func (s *Solver) solveEven() (int, []Square) {
    half := s.boardSize / 2
    return 4, []Square{
        {1, 1, half},
        {1, half + 1, half},
        {half + 1, 1, half},
        {half + 1, half + 1, half},
    }
}

// scaleSolution scales a solution for composite board sizes
func scaleSolution(count int, squares []Square, factor int) (int,
[]Square) {
    scaled := make([]Square, count)
    for i, sq := range squares {
        scaled[i] = Square{
            x:      (sq.x-1)*factor + 1,
            y:      (sq.y-1)*factor + 1,
            size: sq.size * factor,
        }
    }
    return count, scaled
}

// solve finds the optimal square packing for the current board size

```

```

func (s *Solver) solve() (int, []Square) {
    fmt.Printf("Finding optimal solution for square size: %d\n",
s.boardSize)
    // Even-sized boards have a known optimal solution
    if s.boardSize%2 == 0 {
        fmt.Println("N is even, so solution is trivial:\n")
        return s.solveEven()
    }

    // For composite sizes, solve for the smallest prime factor and
scale up
    prime := smallestPrimeFactor(s.boardSize)
    if prime < s.boardSize {
        fmt.Println("N is composite number, so we solve for the
size of the smallest")
        fmt.Printf("prime factor (%d), then scale the found
solution up\n", prime)
        // Create a sub-solver for the prime-sized board
        subSolver := NewSolver(prime)
        count, squares := subSolver.solve()
        return scaleSolution(count, squares, s.boardSize/prime)
    }

    fmt.Printf("Board size %d is prime - performing full search\n",
s.boardSize)

    // Initialize search for prime sizes
    s.bestCount = s.boardSize * s.boardSize
    s.bestSquares = nil
    s.board = make([]int, s.boardSize)

    // Place three initial squares for optimization
    if s.boardSize > 2 {
        size1 := (s.boardSize + 1) / 2
        size2 := (s.boardSize - 1) / 2

        s.place(0, 0, size1, true)
        s.place(0, size1, size2, true)
        s.place(size1, 0, size2, true)

        initialSquares := []Square{
            {1, 1, size1},
            {1, size1 + 1, size2},
            {size1 + 1, 1, size2},
        }

        fmt.Println("Heuristic initial placement:")
        fmt.Printf("  Square 1: size %d at position (0,0)\n",
size1)
        fmt.Printf("  Square 2: size %d at position (0,%d)\n",
size2, size1)
        fmt.Printf("  Square 3: size %d at position (%d,0)\n",
size2, size1)

        s.backtrack(initialSquares, len(initialSquares))
    } else {

```

```

        s.backtrack([]Square{}, 0)
    }

    return s.bestCount, s.bestSquares
}

func smallestPrimeFactor(n int) int {
    for i := 2; i*i <= n; i++ {
        if n%i == 0 {
            fmt.Printf("Found smallest prime factor for number %d
it's %d\n", n, i)
            return i
        }
    }
    fmt.Printf("Found smallest prime factor for number %d it's %d\n",
n, n)
    return n
}

func (s *Solver) printBoard() {
    if len(s.board) == 0 {
        fmt.Println("Board isn't initialised")
        return
    }

    for row := 0; row < s.boardSize; row++ {
        for col := 0; col < s.boardSize; col++ {
            if (s.board[row]>>col)&1 == 1 {
                fmt.Print("■ ")
            } else {
                fmt.Print("□ ")
            }
        }
        fmt.Println()
    }
    fmt.Println()
}

func main() {
    var boardSize int
    fmt.Scan(&boardSize)

    solver := NewSolver(boardSize)
    count, squares := solver.solve()

    fmt.Println(count)
    for _, sq := range squares {
        fmt.Printf("%d %d %d\n", sq.x, sq.y, sq.size)
    }
}

```